

VESPER-SEC-01 — Week 6 Deliverable

First Working Artifact: Cryptographic Primitive Benchmarking Under Cortex-M4 / Renode Emulation

Rutvika Mahadev Ashtekar

VISVAMBHARA, Week 6

Domain: Cybersecurity (Option 3.5 A)

1. Week 6 Objective

Per the charter's weekly cadence (§2.1), Week 6 requires "a minimum version that produces a number, verified against at least one analytical limit." This document reports the first working benchmark artifact built on top of the Week 5 analytical model: four cryptographic primitives compiled for ARM Cortex-M4 and executed under Renode, each producing a measured cycle count via the DWT cycle counter defined in Week 5.

Scope note: published cryptographic-result validation is outside the Week 6 scope and is planned for the later validation stage (Week 8). This document does not claim hardware-validated cycle accuracy for any primitive.

2. Environment

Component	Detail
Toolchain	Arm GNU Toolchain (arm-none-eabi-gcc), MSYS2 supporting build environment
Emulator	Renode, STM32F4 Discovery platform model
Target	ARM Cortex-M4 (STM32F407-class)
Cycle measurement	DWT peripheral mapped at 0xE0001000 (168 MHz); DWT→CYCCNT reset and enabled before each run, per the Week 5 model (Eq. 1)
Workspace	VESPER-SEC-xx/week6/src/{ascon, chacha, kyber-ref, sphincsplus, heinz-mkm4}

3. Analytical Limit Verification (Carried Forward)

Before interpreting the cryptographic measurements, the Week 6 artifact is checked against the simple analytical limiting case established in Week 4/5: 1,000 single-cycle NOP instructions.

Expected	Measured	Discrepancy
1,000 cycles	1,003 cycles	0.3% — within the $\leq 5\%$ tolerance

This confirms the basic instruction-execution/cycle-counting mechanism works. It does not by itself prove cycle-accuracy for multi-cycle, branching cryptographic code — that distinction is kept explicit throughout this document.

4. Benchmark Results

All four target primitives from the Week 5 model were compiled, linked, loaded into Renode, and executed. Each produced a measured cycle count.

4.1 Ascon v1.2

Metric	Value
Measured cycles	9,606 (16-byte plaintext)
Cycles/byte	600.375 (9,606 / 16, per Week 5 Eq. 2)
Reported stack	264 B
ELF size (text/data/bss/total)	22,984 / 0 / 44 / 23,028 B

4.2 ChaCha20-Poly1305

Metric	Value
Measured cycles	40,597 (114-byte plaintext)
Cycles/byte	≈ 356.114 (40,597 / 114)
Reported stack	544 B
ELF size (text/data/bss/total)	5,104 / 0 / 148 / 5,252 B
Validation flags	valid = 1, done = 1

4.3 ML-KEM / Kyber-512

Configured with `KYBER_K = 2` (Kyber-512). For a fixed-size KEM operation, cycles/operation is the primary metric — not cycles/byte — per the Week 5 limiting-case discussion (§8.2 of the Week 5 model).

Operation	Measured Cycles
Key generation	1,157,443
Encapsulation	1,300,194
Decapsulation	1,538,934

Reported stack: 9,748 B. ELF size (text/data/bss/total): 10,992 / 0 / 3,288 / 14,280 B. Validation flags: valid = 1, done = 1.

4.4 SPHINCS+-Haraka-128f

Operation	Measured Cycles
Key generation	153,849,855
Signing	3,738,100,285
Verification	242,116,523

Signature length: 17,088 B. ELF size (text/data/bss/total): 11,592 / 4 / 17,292 / 28,888 B. Validation flags: valid = 1, done = 1.

Limitation: the SPHINCS+ peak-stack measurement was contaminated by execution occurring before the stack-pattern read, so the raw figure (57,856 B) is not reported as a valid peak-stack result. This is recorded as an open limitation, not silently corrected or hidden.

Test and Verification Results

The completed Week 6 benchmark artifacts were checked using a project-specific test suite containing 22 checks. The suite covers the analytical limiting case and result-validity/logic checks for all four benchmarked primitives.

Overall result: 22/22 Week 6 project test-suite checks passing.

- NOP analytical limiting case — PASS
- Ascon v1.2 checks — PASS
- ChaCha20-Poly1305 checks — PASS
- ML-KEM/Kyber-512 checks — PASS
- SPHINCS+-Haraka-128f checks — PASS

The NOP check measured 1,003 cycles against an expected 1,000 cycles, giving a 0.3% discrepancy (consistent with the Section 3 result).

```
1 import math
2
3 TESTS = []
4
5 def check(name, condition, detail):
6     TESTS.append((name, bool(condition), detail))
7
8 nop_expected = 1000
```

```
PS C:\Users\Hp\OneDrive\Desktop\VESPER-SEC-xx\week6> python ".\tests\test_week6_results.py"
[PASS] NOP analytical limit: expected=1000, measured=1003, error=0.3%
[PASS] Ascon completion: result_done=1
[PASS] Ascon cycle result: cycles=9606
[PASS] Ascon stack result: stack=264 B
[PASS] Ascon cycles/byte: 600.375 cycles/byte
[PASS] ChaCha completion: result_done=1
[PASS] ChaCha validity: result_valid=1
[PASS] ChaCha cycle result: cycles=40597
[PASS] ChaCha stack result: stack=544 B
[PASS] ChaCha cycles/byte: 356.114 cycles/byte
[PASS] Kyber completion: result_done=1
[PASS] Kyber validity: result_valid=1
[PASS] Kyber keypair cycles: cycles=1157443
[PASS] Kyber encapsulation cycles: cycles=1300194
[PASS] Kyber decapsulation cycles: cycles=1538934
[PASS] Kyber stack result: stack=9748 B
[PASS] SPHINCS completion: result_done=1
[PASS] SPHINCS validity: result_valid=1
[PASS] SPHINCS keypair cycles: cycles=153849855
[PASS] SPHINCS signing cycles: cycles=3738100285
[PASS] SPHINCS verification cycles: cycles=242116523
[PASS] SPHINCS signature length: signature=17088 B

RESULT: 22/22 passing
PS C:\Users\Hp\OneDrive\Desktop\VESPER-SEC-xx\week6>
```

The test suite verifies Week 6 analytical and logical conditions and the recorded benchmark outputs. It does not claim published-result reproduction or cryptographic cycle-accuracy validation. Published-result validation remains outside the Week 6 scope. The SPHINCS+ peak-stack result is excluded because its measurement was determined to be invalid.

5. SPHINCS+ Source Attribution

The SPHINCS+ reference implementation was taken from the official sphincs/sphincsplus repository and is not claimed as original work. The following integration-level changes were made to run it in the bare-metal Cortex-M4/Renode environment, and are the only parts original to this project:

- Deterministic randombytes() implementation (benchmark/randombytes.c), replacing OS-sourced randomness unavailable in bare-metal.
- Local memset/memcpy/memmove/memcmp implementations (benchmark/mem.c), since no host C runtime is present.
- Minor source compatibility/pointer-type fixes (including in merkle.c).
- A project-specific linker script (sphincs.ld) and Cortex-M4 startup code

(startup.s).

- A dedicated Renode platform file (sphincs-stm32f4.repl) adding the DWT mapping the default STM32F4 platform did not provide.

6. Heinz Reference — Investigated, Not Used for Validation

The Heinz [8] "First-Order Masked Kyber on ARM Cortex-M4" reference was built successfully (ELF: elf/crypto_kem_kyber768_m4_test.elf, 370 KB), after correcting missing .type/.thumb_func metadata in several Keccak assembly functions (KeccakF1600_Initialize, KeccakF1600_StateXORBytes, KeccakF1600_StateExtractBytes, KeccakF1600_StatePermute).

A citation discrepancy was identified during this work: the Week 4 proposal cites Heinz [8] as a masked Kyber-512 benchmark, but Heinz's actual published implementation is masked Kyber768 — a different parameter set from this project's unmasked Kyber-512. Comparing the two cycle counts directly would not be a like-for-like comparison, so no such comparison is made here. Heinz's documented measurement procedure also requires physical STM32F4 Discovery hardware (ST-Link/OpenOCD/GDB), which falls outside this project's Renode-only, zero-hardware approach.

Status: Heinz reference build — complete. Direct cycle-count validation against Heinz was not attempted and is not claimed. The later validation stage can either use a matching Kyber-512 published reference or document the parameter-set mismatch as a scoped limitation.

7. Evidence Collected

Three evidence screenshots were captured per primitive: compile/ELF size, ELF symbols, and Renode execution results (12 screenshots total, plus the Week 4 toolchain-verification set).

Primitive	Evidence Files
Ascon v1.2	01_ascon_final_compile_size.png · 02_ascon_final_elf_symbols.png · 03_ascon_final_renode_results.png
ChaCha20-Poly1305	01_chacha_final_compile_size.png · 02_chacha_final_elf_symbols.png · 03_chacha_final_renode_results.png
Kyber-512	01_kyber_final_compile_size.png · 02_kyber_final_elf_symbols.png · 03_kyber_final_renode_results.png
SPHINCS+-Haraka-128f	01_sphincs_final_compile_size.png · 02_sphincs_final_elf_symbols.png · 03_sphincs_final_renode_results.png

8. What Is Not Claimed (Explicitly Out of Scope for Week 6)

- Published cryptographic-result reproduction (Heinz or otherwise) — this is a Week 8 requirement.
- Hardware-validated cycle accuracy for any of the four primitives.
- A final FEASIBLE/MARGINAL/NOT FEASIBLE classification is not claimed in Week 6.
- A valid SPHINCS+ peak-stack figure — excluded from quantitative comparison for the reason stated in §4.4.

9. Conclusion

The Week 6 requirement — a minimum working artifact producing a number, verified against at least one analytical limit — is satisfied. Four cryptographic primitives (Ascon v1.2, ChaCha20-Poly1305, ML-KEM/Kyber-512, SPHINCS+-Haraka-128f) were compiled for Cortex-M4, executed under Renode, and each produced measured cycle counts, supported by ELF/build and execution evidence. The execution framework itself was sanity-checked against the 1,000-NOP analytical limit (1,003 measured vs. 1,000 expected, 0.3% discrepancy). No claim of published-result reproduction, hardware validation, or final feasibility verdict is made at this stage; those remain explicitly scoped to later weeks.